

人工智慧導論



常用的Java類別與套件

Math數學類別

- **Java API**的**Math**數學類別提供數學常數和各種數學函數的類別方法，我們可以在**Java**程式碼使用亂數、計算最大值、最小值、三角和指數等數學函數，因為它是類別方法，在呼叫方法時需要指明類別**Math**。

計算亂數、最大和最小值

□ Math類別提供方法計算亂數、最大值和最小值：

方法	說明
<code>int max(int, int)</code> <code>long max(long, long)</code> <code>double max(double, double)</code> <code>float max(float, float)</code>	傳回 2 個 int、long、double、float 參數中的最大值
<code>int min(int, int)</code> <code>long min(long, long)</code> <code>double min(double, double)</code> <code>float min(float, float)</code>	傳回 2 個 int、long、double、float 參數中的最小值
<code>double random()</code> <code>float random()</code>	傳回亂數值，其範圍為 0.0~1.0
<code>int round(double)</code> <code>long round(double)</code>	將 double 參數值四捨五入後傳回整數值

□ Random seed

```
Random rand = new Random(seed);  
double random = rand.nextDouble();
```

Math類別的數學常數和方法-常數

- Math類別提供兩個常用的數學常數，其說明如下表所示：

常數	說明
E	自然數 $e=2.718281828459045$
PI	圓周率 $\pi =3.141592653589793$

Math類別的數學常數和方法

- Math類別還提供三角函數（Trigonometric）、指數（Exponential）和對數（Logarithmic）的方法。其相關方法說明如下表所示：

方法	說明
int abs(int) long abs(long) double abs(double) float abs(float)	傳回絕對值
double acos(double)	反餘弦函數
double asin(double)	反正弦函數
double atan(double)	反正切函數
double atan2(double1, double2)	參數 double1/double2 的反正切函數值
double ceil(double)	傳回 double 值大於或等於參數的最小 double 整數
double cos(double)	餘弦函數
double exp(double)	自然數的指數 e^x

Math類別的數學常數和方法

方法	說明
<code>double floor(double)</code>	傳回 <code>double</code> 值小於或等於參數的最大 <code>double</code> 整數
<code>double log(double)</code>	自然對數
<code>double pow(double, double)</code>	傳回第 1 個參數為底，第 2 個參數的次方值
<code>double rint(double)</code>	傳回最接近參數的 <code>double</code> 整數值
<code>double sin(double)</code>	正弦函數
<code>double sqrt(double)</code>	傳回參數的平方根
<code>double tan(double)</code>	正切函數
<code>double toDegrees(double)</code>	傳回參數轉換成的角度（Degree）
<code>double toRadians(double)</code>	傳回參數轉換成的徑度（Radian）

- 上表三角函數的參數是徑度，並不是角度，如果是角度，請使用**toRadians()**方法先轉換成徑度

List介面的基礎-說明

- List介面也是繼承自Collection介面，實作此介面的集合類別是一種「循序集合物件」（**Ordered Collection**），允許儲存重複元素，而且元素擁有索引位置，可以使用類似陣列索引方式來存取元素。
- 「循序集合物件」（**Ordered Collection**）和「排序集合物件」（**Sorted Collection**）並不相同，循序集合物件如同陣列是以元素位置的索引來排列；排序集合物件是以元素值的大小進行排序。
 - 。

List介面的基礎-介面方法

List 介面方法	說明
void add(int, E)	在參數 int 位置插入 E 元素，索引位置是從 0 開始
boolean addAll(int, Collection)	在參數 int 位置插入 Collection 集合物件的所有元素，成功傳回 true
E get(int index)	傳回參數 int 位置的元素
int indexOf(Object)	搜尋參數 Object 的元素，傳回值是第 1 個找到元素的位置，如果沒有找到傳回-1
int lastIndexOf(Object)	反向搜尋參數 Object 的元素，傳回值是最後 1 個找到元素的位置，如果沒有找到傳回-1
E remove(int)	刪除參數 int 位置的元素
E set(int, E)	將參數 int 位置的元素取代成參數 E，傳回集合物件被取代的元素
List<E> subList(int, int)	傳回從第 1 個參數 int（包含）到第 2 個參數 int（不包含）間的子集合

ArrayList類別-說明

- ▣ **ArrayList**類別實作**List**介面，儲存元素的方式類似陣列，元素使用索引位置依序存入，只需將元素新增或插入**ArrayList**物件，並不用事先宣告物件的尺寸，如同一個可調整陣列尺寸的動態陣列。
 -

ArrayList類別的方法

建構子	說明
<code>public ArrayList<E>()</code>	建立空的 ArrayList
<code>public ArrayList<E>(int initialCapacity)</code>	建立含有 initialCapacity 個元素的空 ArrayList
<code>public ArrayList<E>(Collection<? extends E> c)</code>	使用 c 建立 ArrayList

方法	說明
<code>boolean add(E e)</code>	依序增加元素，索引值遞增
<code>E remove(int index)</code>	移除索引值所在的元素
<code>E get(int index)</code>	依索引值 index 取得元素
<code>int size()</code>	取得大小
<code>void clear()</code>	清空所有元素

ArrayList類別的範例

■ Example:

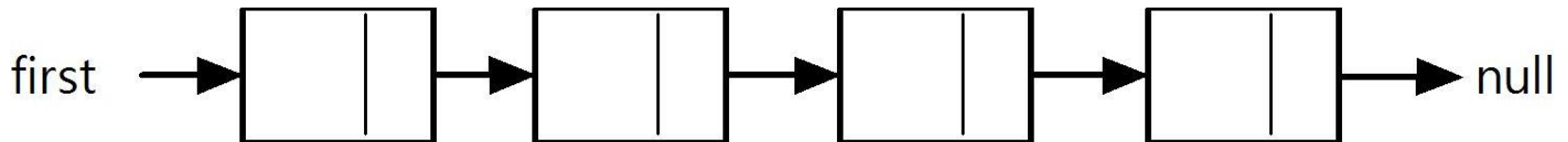
```
public static void main(String[] args) {  
    Scanner cin = new Scanner(System.in);  
    ArrayList<Integer> list = new ArrayList<Integer>();  
    int x;  
    while ((x = cin.nextInt()) != 0) {  
        list.add(x);  
    }  
  
    System.out.println("total " + list.size() + " input elements.");  
    System.out.println(list.toString());  
    System.out.println("2nd element = " + list.get(2));  
}
```

```
3 5 7  
1 0
```

```
total 4 input elements.  
[3, 5, 7, 1]  
2nd element = 7
```

LinkedList類別-說明

- **LinkedList**類別是實作**List**介面，這是類似「鏈結串列」（**Linked Lists**）資料結構的類別，各節點如同火車掛車廂一般，將每一個車廂的節點連接起來，如下圖所示：



LinkedList類別的方法

建構子	說明
<code>public LinkedList<E>()</code>	建立空的 LinkedList
<code>public LinkedList<E>(Collection<? extends E> c)</code>	使用 c 建立 LinkedList

方法	說明
<code>boolean add(E e)</code>	將元素加到最後一個
<code>void add(int index, E element)</code>	將元素加到索引值位置
<code>E remove(int index)</code>	移除索引值所在的元素
<code>E get(int index)</code>	依索引值 index 取得元素
<code>int size()</code>	取得大小
<code>void clear()</code>	清空所有元素

實作Set介面的集合類別

- ▣ **Set**介面是**Collection**介面的子介面，實作**Set**介面的集合物件，其儲存元素並沒有重複，如果是實作子介面**SortedSet**的集合物件，儲存的就是排序元素。

Set介面的基礎-介面方法

Set 介面方法	說明
<code>boolean isEmpty()</code>	檢查集合物件是否是空的，傳回 <code>true</code> 為是
<code>int size()</code>	傳回集合物件目前的元素數
<code>boolean add(E)</code>	新增參數 <code>E</code> 的元素，成功傳回 <code>true</code>
<code>boolean addAll(Collection)</code>	將參數 <code>Collection</code> 集合物件的元素都新增為元素，成功傳回 <code>true</code>
<code>boolean remove(Object)</code>	刪除參數 <code>Object</code> 的元素，如果成功表示擁有此元素，傳回 <code>true</code>
<code>boolean removeAll(Collection)</code>	刪除集合物件中擁有參數 <code>Collection</code> 集合物件的元素，成功傳回 <code>true</code>
<code>boolean retainAll(Collection)</code>	保留集合物件中擁有參數 <code>Collection</code> 集合物件的元素，成功傳回 <code>true</code>
<code>void clear()</code>	清除集合物件的所有元素
<code>boolean contains(Object)</code>	檢查集合物件中是否擁有參數 <code>Object</code> 的元素，如果有，傳回 <code>true</code>
<code>boolean containsAll(Collection)</code>	檢查集合物件中是否有參數 <code>Collection</code> 集合物件的元素，如果有，傳回 <code>true</code>

HashSet類別-說明

- **HashSet**類別實作**Set**介面，繼承**Collection**介面的方法，使用「雜湊表」（**Hash Table**）演算法來改進新增、刪除和存取集合物件元素的執行效率，其儲存元素的排列和插入順序不同，也不保證擁有固定的排列順序。

HashSet類別的方法

建構子	說明
HashSet()	建立空的 HashSet, 預設capacity (16) 及 load factor (0.75).
HashSet(Collection<? extends E> c)	建立新的HashSet, 包含特殊集合元素
HashSet(int initialCapacity)	建立空的 HashSet, 自訂 capacity 及預設load factor (0.75).
HashSet(int initialCapacity, float loadFactor)	建立空的 HashSet, 自訂capacity 及 load factor

方法	說明
boolean add(E e)	加入元素(如果不存在)
boolean remove(Object o)	移除元素
boolean contains(Object o)	是否包含某元素
int size()	取得大小
void clear()	清空所有元素

TreeSet類別-說明

- ▣ **TreeSet**類別實作**SortedSet**介面，這是**Set**介面的子介面，其儲存元素不只沒有重複，而且由小到大進行排序，這是一種「排序集合物件」（**Sorted Collection**）。

TreeSet類別-介面方法

- TreeSet類別實作SortedSet介面，除了繼承Set介面的方法外，其新增的介面方法，如下表所示

SortedSet 介面方法	說明
E first()	傳回集合物件的第 1 個元素
E last()	傳回集合物件的最後 1 個元素
SortedSet headSet(E)	傳回小於參數 E 元素的部分 TreeSet 物件
SortedSet<E> subSet(E1, E2)	傳回大於等於參數 E1 元素，到小於 E2 元素的部分 TreeSet 物件
SortedSet<E> tailSet(E)	傳回大於等於參數 E 元素的部分 TreeSet 物件

TreeSet類別的範例

■ Example:

```
public static void main(String[] args) {  
    Scanner cin = new Scanner(System.in);  
    Set<Integer> s = new TreeSet<Integer>();  
    while (cin.hasNextInt()) {  
        s.add(cin.nextInt());  
    }  
  
    for (Iterator iter = s.iterator(); iter.hasNext();) {  
        System.out.print(iter.next() + " ");  
    }  
    System.out.println();  
  
    System.out.println(s.toString());  
    System.out.println(s.contains(0));  
}
```

• Input

3 5 7 1

• Output

1 3 5 7

[1, 3, 5, 7]

false

Map介面的基礎-說明

- **Map**介面是類別架構的根介面，擁有子介面**SortedMap**介面，實作**Map**介面的集合類別，其儲存物件共有**2**個：鍵值（**Key**）和對應值（**Value**）的物件，其中鍵值不可重複，一個鍵值對應一個值。

Map介面的基礎-介面方法

Set 介面方法	說明
<code>boolean isEmpty()</code>	檢查集合物件是否空的，傳回 <code>true</code> 為是
<code>int size()</code>	傳回集合物件目前的元素數
<code>V put(K key, V value)</code>	新增參數 <code>key</code> 的鍵值和 <code>value</code> 值的元素，如果相同的 <code>key</code> 鍵值存在，就是取代 <code>value</code> 值，
<code>void putAll(Map<K,V>)</code>	新增參數 <code>Map</code> 集合物件的元素，成功傳回 <code>true</code>
<code>V remove(Object key)</code>	刪除參數 <code>key</code> 鍵值的元素，如果 <code>key</code> 鍵值不存在傳回 <code>null</code> ，否則傳回被刪除的 <code>value</code> 值
<code>void clear()</code>	清除集合物件的所有元素
<code>boolean containsKey(Object)</code>	檢查集合物件中是否擁有參數 <code>Object</code> 的鍵值，如果有傳回 <code>true</code>
<code>boolean containsValue(Object)</code>	檢查集合物件中是否擁有參數 <code>Object</code> 的值，如果有傳回 <code>true</code>
<code>V get(Object key)</code>	傳回參數 <code>key</code> 的鍵值所對應的值
<code>Set<K> keySet()</code>	將鍵值轉換成實作 <code>Set</code> 介面的集合物件
<code>Collection<V> values()</code>	將值轉換成實作 <code>Collection</code> 介面的集合物件

HashMap類別-說明

- **HashMap**類別是實作**Map**介面，其功能相當於是舊版**Hashtable**類別，儲存元素有2個：**key**鍵值和其對應值，並且允許儲存**null**值。
- **HashMap**類別提供數種過載的建構子，在Java程式建立**HashMap**物件，如下所示：

```
HashMap<String,String> hmap = new  
    HashMap<>();
```
- 上述程式碼建立**HashMap**物件，因為儲存元素有鍵值和對應值，所以指定2個泛型型態。

HashMap類別-建構子

- 過載建構子的說明，如下表所示：

建構子	說明
HashMap<K,V>()	建立空的 HashMap 物件，預設的元素容量是 16
HashMap<K,V>(int)	建立空的 HashMap 物件，可以指定擁有參數 int 的元素數
HashMap<K,V>(Map<K,V>)	使用參數 Map 物件的元素建立的 HashMap 物件

TreeMap類別-說明

- TreeMap類別實作SortedMap介面，SortedMap介面是Map介面的子介面，實作此介面的集合物件是以鍵值由小到大進行排序。在Java程式建立TreeMap物件，如下所示：

```
TreeMap<String,String> tmap = new  
    TreeMap<>();
```

- 上述程式碼建立TreeMap物件，元素都為字串String資料型態。

TreeMap類別-建構子

- 過載建構子的說明，如下表所示：

建構子	說明
<code>TreeMap<K,V>()</code>	建立空的 <code>TreeMap</code> 物件，使用鍵值的大小進行排序
<code>TreeMap<K,V>(Map<K,V>)</code>	使用參數 <code>Map</code> 物件的元素建立 <code>TreeMap</code> 物件
<code>TreeMap<K,V>(Comparator c)</code>	建立空的 <code>TreeMap</code> 物件，以參數 <code>Comparator</code> 比較物件進行排序
<code>TreeMap<K,V>(SortedMap<K,V>)</code>	使用參數實作 <code>SortedMap</code> 介面的物件來建立 <code>TreeMap</code> 物件，其排序方式和參數相同

TreeMap類別-介面方法

- TreeMap類別實作SortedMap介面，除了繼承Map介面的方法外，其新增的介面方法，如下表所示：

SortedMap 介面方法	說明
K firstKey()	傳回集合物件的第 1 個，即最小的鍵值
K lastKey()	傳回集合物件的最後 1 個，即最大的鍵值
SortedMap<K,V> headMap(K)	傳回小於參數 K 鍵值的部分 TreeMap 物件
SortedMap<K,V> subMap(K1, K2)	傳回大於等於參數 K1 鍵值，到小於 K2 鍵值的部分 TreeMap 物件
SortedMap<K,V> tailMap(K)	傳回大於等於參數 K 鍵值的部分 TreeMap 物件

Stack類別的方法

建構子	說明
Stack()	建立空的 Stack

方法	說明
boolean empty()	測試是否為空
E peek()	偷看stack頂端的物件，但不移除
E pop()	取得Stack頂端的物件，並將其移除
E push (E item)	將物件塞進Stack頂端
int search (Object o)	回傳物件存在位置與頂端的相對距離， 如果不在，則回傳-1

Queue類別的方法

建構子	說明
Queue()	建立空的 Queue

方法	說明
boolean offer (E e)	塞入元素
E poll ()	拉出元素

Iterator介面輸出元素-說明

- ❑ **Iterator**介面改進舊版**Java**技術的**Enumeration**介面，提供一致方法來走訪集合物件的元素或刪除元素，因為**Collections**集合物件已經實作**Iterator**介面，換句話說，只需使用**Collection**、**Set**和**List**介面的**iterator()**方法，就可以取得**Iterator<E>**介面的物件，如下表所示：

介面方法	說明
<code>Iterator<E> iterator()</code>	取得集合物件實作 <code>Iterator<E></code> 介面的物件

Iterator介面輸出元素-範例

- 例如：**HashSet<String>**物件**hset**可以使用上表方法取得**Iterator**介面物件，如下所示：

```
Iterator<String> iterator =  
    hset.iterator();
```

- 上述程式碼將**HashSet**物件的元素轉換成**Iterator<E>**介面物件，泛型型態為**String**，現在我們只需呼叫介面方法，就可以依序走訪和刪除元素。

Iterator介面輸出元素-介面方法

Iterator 介面方法	說明
boolean hasNext()	是否有下一個元素，如果有，傳回 true
E next()	取得集合物件的下一個元素
void remove()	刪除最後一個取得的元素

- 在取得**Iterator<E>**介面物件後，即可配合**while**迴圈走訪集合物件的元素，如下所示：

```
while (iterator.hasNext())
```

```
    System.out.print(" "+iterator.next());
```

- 上述程式碼使用**hasNext()**方法檢查是否有下一個元素，如果有，使用**next()**方法取得此元素。

Java Algorithms

SN	Methods with Description
1	static int binarySearch(List list, Object value, Comparator c) Searches for value in list ordered according to c. Returns the position of value in list, or -1 if value is not found
2	static int binarySearch(List list, Object value) Searches for value in list. The list must be sorted. Returns the position of value in list, or -1 if value is not found.
3	static void copy(List list1, List list2) Copies the elements of list2 to list1.
4	static Enumeration enumeration(Collection c) Returns an enumeration over c.
5	static void fill(List list, Object obj) Assigns obj to each element of list.
6	static int indexOfSubList(List list, List subList) Searches list for the first occurrence of subList. Returns the index of the first match, or .1 if no match is found.
7	static int lastIndexOfSubList(List list, List subList) Searches list for the last occurrence of subList. Returns the index of the last match, or .1 if no match is found.

Java Algorithms

SN	Methods with Description
8	static ArrayList list(Enumeration enum) Returns an ArrayList that contains the elements of enum.
9	static Object max(Collection c, Comparator comp) Returns the maximum element in c as determined by comp.
10	static Object max(Collection c) Returns the maximum element in c as determined by natural ordering. The collection need not be sorted.
11	static Object min(Collection c, Comparator comp) Returns the minimum element in c as determined by comp. The collection need not be sorted.
12	static Object min(Collection c) Returns the minimum element in c as determined by natural ordering.
13	static List nCopies(int num, Object obj) Returns num copies of obj contained in an immutable list. num must be greater than or equal to zero.
14	static boolean replaceAll(List list, Object old, Object new) Replaces all occurrences of old with new in list. Returns true if at least one replacement occurred. Returns false, otherwise.

Java Algorithms

SN	Methods with Description
15	static void reverse(List list) Reverses the sequence in list.
16	static Comparator reverseOrder() Returns a reverse comparator
17	static void rotate(List list, int n) Rotates list by n places to the right. To rotate left, use a negative value for n.
18	static void shuffle(List list, Random r) Shuffles (i.e., randomizes) the elements in list by using r as a source of random numbers.
19	static void shuffle(List list) Shuffles (i.e., randomizes) the elements in list.
20	static Set singleton(Object obj) Returns obj as an immutable set. This is an easy way to convert a single object into a set.
21	static List singletonList(Object obj) Returns obj as an immutable list. This is an easy way to convert a single object into a list.

Java Algorithms

SN	Methods with Description
22	static Map singletonMap(Object k, Object v) Returns the key/value pair k/v as an immutable map. This is an easy way to convert a single key/value pair into a map.
23	static void sort(List list, Comparator comp) Sorts the elements of list as determined by comp.
24	static void sort(List list) Sorts the elements of list as determined by their natural ordering.
25	static void swap(List list, int idx1, int idx2) Exchanges the elements in list at the indices specified by idx1 and idx2.
26	static Collection synchronizedCollection(Collection c) Returns a thread-safe collection backed by c.
27	static List synchronizedList(List list) Returns a thread-safe list backed by list.
28	static Map synchronizedMap(Map m) Returns a thread-safe map backed by m.

Java Algorithms-Examples

■ Example:

```
String[] arr1 = {"ABCD", "BADC", "DCBA", "ADCB"};  
System.out.println(Arrays.toString(arr1));
```

```
Arrays.sort(arr1);  
System.out.println(Arrays.toString(arr1));
```

```
System.out.printf("index of %s is %d\n", "BADC", Arrays.binarySearch(arr1, "BADC"));
```

```
Collections.reverse(Arrays.asList(arr1));  
System.out.println(Arrays.toString(arr1));
```

```
System.out.printf("index of %s is %d\n", "BADC", Arrays.binarySearch(arr1, "BADC"));
```

```
[ABCD, BADC, DCBA, ADCB]  
[ABCD, ADCB, BADC, DCBA]  
index of BADC is 2  
[DCBA, BADC, ADCB, ABCD]  
index of BADC is 1
```